MANUAL DE IMPLEMENTACIÓN Y OPERACIÓN

Gestión de usuarios Linux con Ansible y AWX

Crear cuentas, cambiar contraseñas con política «nunca caduca», auditar permisos sudo y dar de baja usuarios, todo desde encuestas de AWX.

1
Crear usuarios
con contraseña o con llave SSH

2
Cambiar contraseña
con opción «nunca caduca»

3
Auditoría de usuarios
y permisos sudo

4
Bloquear o eliminar
usuarios

Proyecto **gestion_usuarios** · Versión 1.0 · 10/11/2026
Linux (RHEL / Oracle / SLES / Ubuntu) y Solaris 11

Contenido

1 Qué hace el proyecto 3
2 Cómo funciona una ejecución 4
3 Antes de empezar 5
4 Estructura del proyecto 6
5 Configurar AWX paso a paso 8
Paso 1 · Subir el proyecto a Git 8
Paso 2 · Credenciales 8
Paso 3 · Proyecto 10
Paso 4 · Inventario 11
Paso 5 · Plantillas de trabajo 11
Paso 6 · Encuestas 12
Paso 7 · Permisos 13
Paso 8 · Workflows con aprobación 14
6 Uso diario: cómo diligenciar cada opción 18
Opción 1 · Crear usuarios 18
Opción 2 · Cambiar contraseña 22
Opción 3 · Auditoría 23
Opción 4 · Bloquear o eliminar 24
Lanzar y aprobar un workflow 24
7 Cómo leer la auditoría 27
8 Seguridad y buenas prácticas 31
9 Solución de problemas 32
10 Uso por línea de comandos 33
A Anexo A · Todas las preguntas de las encuestas 34
B Anexo B · Grupos del inventario 39
C Anexo C · Cambios frente a los proyectos anteriores 40

1 Qué hace el proyecto

Este proyecto reúne en un solo lugar la gestión de cuentas de los servidores Linux. Cada operación es una **plantilla de AWX con su propia encuesta**: el operador llena un formulario y AWX hace el trabajo en los servidores elegidos.

1 Crear usuarios

Alta de una o varias cuentas con **contraseña inicial** o con **llave SSH**. Define grupo, grupos secundarios y sudo: ninguno, completo o limitado a ciertos comandos.

`01_crear_usuarios.yml`

2 Cambiar contraseña

Cambia la clave de cualquier usuario (incluido **root**). Puede dejarla **sin caducidad** para que el hardening no la venza, forzar el cambio en el próximo ingreso o desbloquear la cuenta.

`02_cambiar_contrasena.yml`

3 Auditoría

Lista las cuentas con UID ≥ 1000 y dice si tienen sudo **completo, limitado o ninguno**, con qué comandos, si es sin contraseña y de dónde sale la regla. Entrega reporte HTML y CSV.

`03_auditoria_usuarios.yml`

4 Bloquear o eliminar

Bloqueo reversible (conserva el home) o eliminación con respaldo del home. Para eliminar hay que escribir CONFIRMAR.

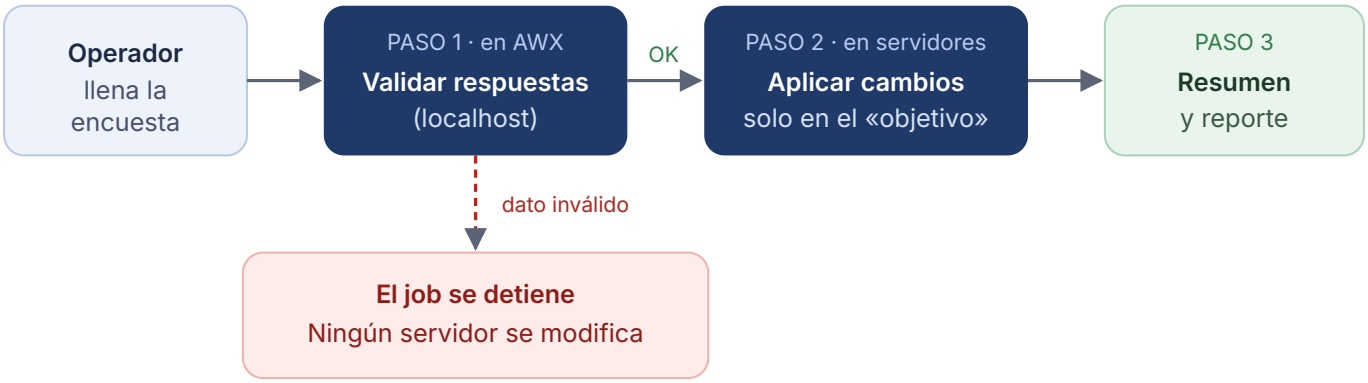
`04_bloquear_eliminar_usuarios.yml`

Protecciones incluidas

- **Nada se ejecuta sin «objetivo».** Si la encuesta no indica servidores o grupo, el job no toca ningún servidor.
- **Cuentas protegidas.** root, las cuentas del sistema y las que usa AWX para conectarse (`ansible_test`, `adminso`, `tux`, `azureuser`) no se pueden crear, bloquear ni eliminar.
- **Cuentas de servicio intactas.** Por debajo del UID 1000 no se cambian contraseñas ni se eliminan cuentas, salvo autorización explícita.
- **Contraseñas protegidas.** AWX las guarda cifradas y no aparecen en la salida del job.
- **sudo validado.** Cada regla se revisa con `visudo` antes de instalarla: una regla mal escrita no se aplica.
- **Una llave por usuario, en todos los servidores.** Las llaves viven en `keys/`: si el usuario ya tiene, se reutiliza; solo se genera una nueva si no tiene o si se marca la opción crítica «Renovar llave».
- **La auditoría es de solo lectura.** No cambia nada en los servidores.

2 Cómo funciona una ejecución

Las cuatro opciones siguen el mismo recorrido de tres pasos. Lo importante: **primero se revisan las respuestas** y solo si todo está bien se entra a los servidores.



Paso	Dónde ocurre	Qué pasa
1 · Validar	Dentro de AWX	Se revisan formato de usuarios, contraseña y confirmación, llaves SSH, comandos sudo y cuentas protegidas. Si algo está mal, el job falla con un mensaje que explica qué corregir.
2 · Aplicar	Servidores del «objetivo»	Se ejecutan los cambios en cada servidor del grupo o lista indicada. Cada servidor se procesa por separado: si uno falla, los demás continúan.
3 · Resumir	Salida del job	Cada servidor muestra qué se hizo (creados, ya existentes, contraseñas cambiadas). La auditoría además genera reportes HTML y CSV.

El campo «Servidores o grupo del inventario»
Acepta un grupo (`GEOPOS` , `PPU` , `OCI_PEOPLE_PRD` ...), un servidor (`ktuno-n0`), varios separados por coma (`srv1,srv2`) o combinaciones: `grupo1:grupo2` (ambos grupos) y `grupo:!servidor` (el grupo excepto un servidor). Respete mayúsculas y minúsculas.

3 Antes de empezar

Tres puntos que conviene revisar antes de publicar el proyecto en AWX.

Acción 1 · Proteger el repositorio: contiene las llaves privadas
La carpeta `keys/` (antes `con_LLave/keys/`) es el almacén oficial de llaves: 10 usuarios con su llave privada, pública y `.ppk`. Por eso el repositorio debe ser **privado**, con acceso solo para el equipo de infraestructura, y el token con permiso de escritura debe usarlo únicamente AWX (credencial «Git - publicar llaves», Paso 2.4).

Acción 2 · Revisar los servidores con conexión local
En `inventario/hosts.ini` `PASBOGDIS01` tiene `ansible_connection=local`. En AWX eso no significa «ese servidor»: significa **dentro del contenedor de AWX**, así que al incluir su grupo el job fallará en ese host (o auditará el contenedor). Para usarlo desde AWX quite esa variable y deje su `ansible_host`, o exclúyalo en el objetivo: `PPU:!PASBOGDIS01`.

Acción 3 · Revisar el usuario de conexión por grupo
El usuario con el que AWX entra a cada servidor sale de `inventario/group_vars/` (`ansible_test` en GEOPOS, PPU, PPU_GEOPOS e INFRA2_0; `adminso` en OCI_PEOPLE; `tux` en manhatan) o del propio `hosts.ini` (`tux` en hardening_linux). Ese valor **tiene prioridad** sobre el usuario de la credencial. AZURE y GEOPOS_MARCAR no definen usuario: usan el de la credencial. El detalle por grupo está en el **Anexo B**.

Requisitos

Dónde	Qué se necesita
AWX	AWX 21 o superior, o Ansible Automation Platform 2.x. Execution Environment con ansible-core 2.14 o superior (el predeterminado `awx-ee` sirve).
Colecciones	AWX instala solo `ansible.posix`, `community.crypto` y `community.general` al sincronizar el proyecto. Necesita salida a Galaxy o una credencial de Automation Hub. Sin Internet: use un EE propio que ya las incluya.
Servidores	La cuenta de conexión debe poder usar sudo (sin contraseña, o con la contraseña en la credencial). Python 2.7 o 3.x, que Ansible ya exige.
Sistemas	RHEL / Oracle / Rocky 7-9, SLES 12/15, Ubuntu/Debian y Solaris 11. Solaris está programado pero no se ha probado en un equipo real: valídelo primero en uno de prueba.

4 Estructura del proyecto

Los playbooks quedan en la raíz del repositorio (AWX los busca ahí). La lógica vive en roles y los datos editables en `vars/`, `files/` e `inventario/`.

```
gestion_usuarios/
├── 01_crear_usuarios.yml              ← opción 1
├── 02_cambiar_contrasena.yml          ← opción 2
├── 03_auditoria_usuarios.yml          ← opción 3
├── 04_bloquear_eliminar_usuarios.yml  ← opción 4
├── ansible.cfg
├── collections/requirements.yml       ← colecciones que AWX instala solo
├── inventario/
│   ├── hosts.ini                      ← INVENTARIO 2026 (407 servidores)
│   └── group_vars/                    ← usuario de conexión por grupo
├── vars/
│   ├── comunes.yml                    ← cuentas protegidas, UID mínimo
│   └── catalogos/                     ← listas de usuarios reutilizables
├── keys/                              ← llaves por usuario: <usuario>, .pub y .ppk
│   └── _anteriores/                   ← llaves reemplazadas por una renovación
├── roles/                             ← la lógica de cada opción
├── awx/
│   ├── encuestas/*.json               ← encuestas listas para importar
│   ├── generar_encuestas.py           ← fuente única de las encuestas
│   └── configurar_awx.yml             ← crea todo en AWX automáticamente
└── docs/                              ← este manual
```

¿Qué archivo toco para...?

Necesito...	Archivo	Después
Agregar una lista de usuarios reutilizable (p. ej. una zona nueva)	`vars/catalogos/<nombre>.yml`	Agregar el nombre en `awx/generar_encuestas.py`, ejecutarlo y recargar la encuesta 1.
Cargar la llave que ya tiene un usuario	`keys/<usuario>`, `keys/<usuario>.pub` (y `.ppk`)	Commit. La opción 1 la usará en vez de generar una.
Proteger una nueva cuenta de conexión de AWX	`vars/comunes.yml` → `usuarios_protegidos`	Hacer commit; AWX lo toma al siguiente job.
Cambiar una pregunta de una encuesta	`awx/generar_encuestas.py`	Ejecutarlo, recargar la encuesta y regenerar este manual.
Agregar servidores o grupos	`inventario/hosts.ini` (y `group_vars/<grupo>.yml` si el grupo usa otro usuario de conexión)	Commit y sincronizar el inventario en AWX.

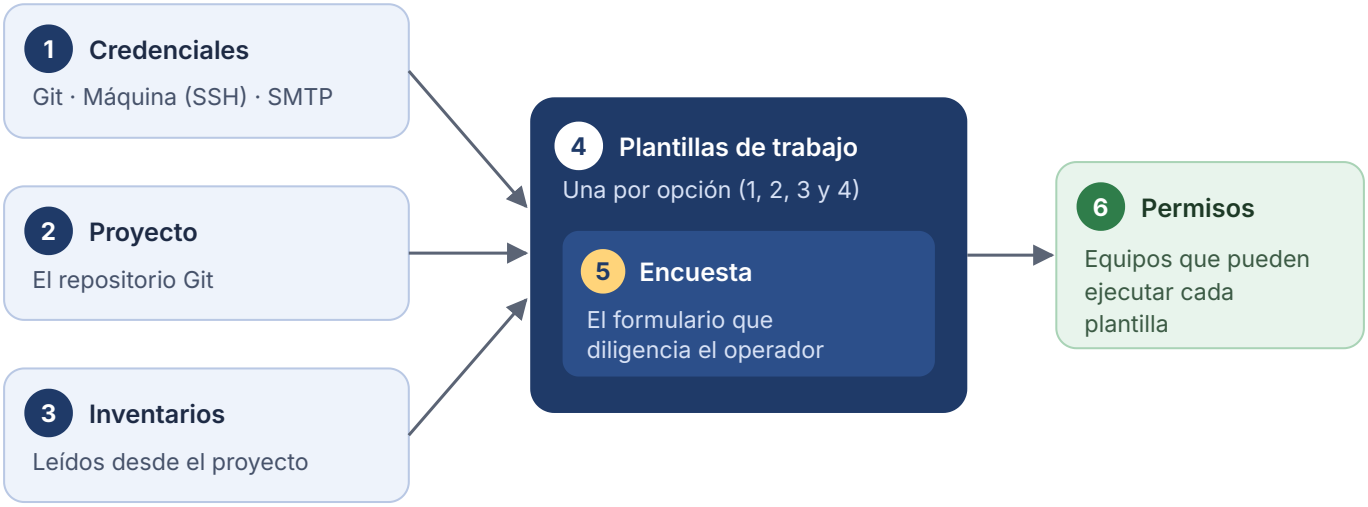
Formato de un catálogo de usuarios

```
catalogo_defaults:          # aplica a todos (opcional)
  grupo: soporte
usuarios:
  - usuario: jperez
    nombre: "Juan Pérez"
  - usuario: jmonitor
    nombre: "Usuario de monitoreo"
    sudo: limitado            # ninguno | completo | limitado
    sudo_nopasswd: true
    sudo_comandos:
      - /usr/bin/systemctl restart httpd
      - /usr/bin/tail -n 200 /var/log/messages
```

Campos opcionales por usuario: `grupo`, `grupos`, `uid`, `shell`, `home`, `llave_publica`, `sudo`, `sudo_nopasswd`, `sudo_comandos`. Lo que se define en el usuario reemplaza lo de `catalogo_defaults` y lo de la encuesta.

5 Configurar AWX paso a paso

AWX necesita seis piezas, que se crean en este orden. Las tres de la izquierda alimentan a cada plantilla; la encuesta va dentro de la plantilla; los permisos dicen quién puede usarla.



¿Prefiere hacerlo automático?
El playbook `awx/configurar_awx.yml` crea el proyecto, los inventarios, el tipo de credencial SMTP y las cuatro plantillas con sus encuestas (ver el final del Paso 6). Se recomienda hacerlo a mano la primera vez para entender cada pieza.

Paso 1 · Subir el proyecto a Git

1. Cree un repositorio (GitLab, GitHub, Bitbucket, Gitea...).
2. Suba **el contenido** de la carpeta `gestion_usuarios/` a la **raíz** del repositorio, de modo que `01_crear_usuarios.yml` quede en la raíz.
3. Verifique que no se suban llaves privadas: el archivo `.gitignore` ya excluye `keys/`, `llaves_generadas/`, `*.ppk` y los reportes.
4. Corrija los conflictos de IP del Anexo B y haga commit.

Paso 2 · Credenciales

Menú **Resources → Credentials → Add**.

2.1 Acceso al repositorio (solo si es privado)

Campo en AWX	Qué poner
Name	`Git - gestion_usuarios`
Credential Type	Source Control
Username / Password o SCM Private Key	Usuario de servicio o *deploy key* de solo lectura

2.2 Conexión a los servidores

Campo en AWX	Qué poner
Name	`Linux - ansible_test`
Credential Type	Machine
Username	`ansible_test`
SSH Private Key	La llave privada de la cuenta de automatización
Privilege Escalation Method	`sudo`
Privilege Escalation Password	Solo si sudo pide contraseña

> **Si cada grupo usa una cuenta distinta**
> Cree una credencial Machine por cuenta (`adminso` , `tux` …) y use plantillas o inventarios separados por grupo, o elimine `ansible_user` de los `group_vars` y deje que lo defina la credencial.

2.3 Correo para la auditoría (opcional)

Primero cree el **tipo** de credencial en **Administration → Credential Types → Add**, con nombre `SMTP - Reportes gestion_usuarios` :

Input configuration

```
fields:
  - {id: smtp_servidor, type: string,
     label: Servidor SMTP}
  - {id: smtp_puerto, type: string,
     label: Puerto, default: "25"}
  - {id: smtp_usuario, type: string,
     label: Usuario (opcional)}
  - {id: smtp_clave, type: string,
     label: Contraseña, secret: true}
  - {id: smtp_seguridad, type: string,
     label: Seguridad, default: try}
  - {id: smtp_remitente, type: string,
     label: Remitente}
required: [smtp_servidor]
```

Injector configuration

```
extra_vars:
  smtp_servidor: '{{ smtp_servidor }}'
  smtp_puerto: '{{ smtp_puerto |
default("25") }}'
  smtp_usuario: '{{ smtp_usuario |
default("") }}'
  smtp_clave: '{{ smtp_clave | default("")
}}'
  smtp_seguridad: '{{ smtp_seguridad |
                    default("try") }}'
  auditoria_correo_remitente:
    '{{ smtp_remitente |
       default("ansible-awx@empresa.local")
}}'
```

Después cree la credencial `SMTP - Correo corporativo` de ese tipo con los datos del servidor de correo.

2.4 Publicar llaves nuevas en Git (necesaria para el método llave)

Cuando la opción 1 genera una llave (usuario nuevo o renovación), AWX la sube a `keys/` del repositorio. Así la siguiente ejecución, en cualquier servidor, **reutiliza esa misma llave**. Sin esta credencial, AWX se niega a generar llaves, porque se perderían al terminar el job.

Credential Type → Input configuration

```
fields:
  - {id: llaves_git_url, type: string,
     label: URL HTTPS del repositorio}
  - {id: llaves_git_rama, type: string,
     label: Rama, default: main}
  - {id: llaves_git_usuario, type: string,
     label: Usuario}
  - {id: llaves_git_token, type: string,
     label: Token de escritura, secret: true}
required: [llaves_git_url,
llaves_git_usuario,
           llaves_git_token]
```

Injector configuration

```
extra_vars:
  llaves_git_url: '{{ llaves_git_url }}'
  llaves_git_rama: '{{ llaves_git_rama |
                       default("main") }}'
  llaves_git_usuario: '{{ llaves_git_usuario
}}'
  llaves_git_token: '{{ llaves_git_token }}'
```

Campo en AWX	Qué poner
Tipo (Administration → Credential Types)	`Git - publicar llaves`, con la configuración de arriba
Credencial (Resources → Credentials)	`Git - publicar llaves gestion_usuarios`
URL HTTPS del repositorio	La misma del proyecto, en HTTPS (p. ej. `https://git.empresa.local/infra/gestion_usuarios.git`)
Usuario / Token	Cuenta de servicio con permiso de **push** en la rama (en GitLab: Project Access Token con `write_repository`; en GitHub: token con *Contents: write*)

> **Rama protegida**
> Si la rama `main` está protegida, permita el push a esa cuenta de servicio o el job fallará con «push rechazado» sin tocar servidores.

Paso 3 · Proyecto

Menú **Resources → Projects → Add**.

Campo en AWX	Qué poner
Name	`Gestion de usuarios Linux`

Campo en AWX	Qué poner
Organization	La suya (p. ej. `Default`)
Execution Environment	El predeterminado o el suyo
Source Control Type	Git
Source Control URL	URL del repositorio
Source Control Branch	`main`
Source Control Credential	`Git - gestion_usuarios` (si aplica)
Options	☑ Clean · ☑ Update Revision on Launch

Guarde y espere a que la sincronización termine en verde. Si falla instalando colecciones, revise el acceso a Galaxy (capítulo 3).

Paso 4 · Inventario

Todo el proyecto usa un solo archivo: `inventario/hosts.ini` (INVENTARIO 2026, 407 servidores en 10 grupos).

1. **Resources → Inventories → Add → Add inventory**. Name: `Inventario 2026`. Guardar.
2. Pestaña **Sources → Add**: Name `Inventario 2026 - desde proyecto`, Source **Sourced from a Project**, Project `Gestion de usuarios Linux`, Inventory file `inventario/hosts.ini`.
3. Marque ☑ Overwrite, ☑ Overwrite variables y ☑ Update on launch. Guardar y pulsar **Sync**.
4. Compruebe en **Hosts** que aparezcan los 407 servidores y en **Groups** los grupos (`GEOPOS`, `PPU`, `OCI_PEOPLE_PRD`...).

> **Variables por grupo**
> Los archivos de `inventario/group_vars/` se cargan solos porque están junto a los `.ini`.

Paso 5 · Plantillas de trabajo

Menú **Resources → Templates → Add → Add job template**. Cree las cuatro con esta configuración común:

Campo en AWX	Qué poner
Job Type	Run
Inventory	`Inventario 2026`
Project	`Gestion de usuarios Linux`

Campo en AWX	Qué poner
Credentials	`Linux - ansible_test`. En la plantilla 1 agregue `Git - publicar llaves gestion_usuarios`; en la 3, `SMTP - Correo corporativo`
Limit	Vacío, con ☑ Prompt on launch (opcional)
Verbosity	0 (Normal)
Forks	20
Options	☑ **Privilege Escalation** · ☐ Concurrent jobs

Name (nombre de la plantilla)	Playbook
USUARIOS | 1 - Crear usuarios	`01_crear_usuarios.yml`
USUARIOS | 2 - Cambiar contraseña	`02_cambiar_contrasena.yml`
USUARIOS | 3 - Auditoría	`03_auditoria_usuarios.yml`
USUARIOS | 4 - Bloquear o eliminar	`04_bloquear_eliminar_usuarios.yml`

Paso 6 · Encuestas

Cada plantilla lleva su encuesta. Las preguntas exactas están en el **Anexo A** y en `awx/encuestas/*.json`. Elija una de estas tres formas de cargarlas:

Forma	Cuándo usarla	Cómo
A · A mano en la UI	Para entender cada pregunta	En la plantilla: pestaña **Survey → Add**. Cree las preguntas del Anexo A en orden, con el nombre de **variable exacto**. Al final active **Survey Enabled**.
B · Por API	Rápido y sin errores de digitación	Ver comandos abajo.
C · Todo como código	Para recrear AWX completo	Ejecutar `awx/configurar_awx.yml` (ver abajo).

> **Consejos para la UI**
> En opción múltiple escriba una opción por línea y marque la predeterminada. En las áreas de texto suba *Maximum length* a 20000 para poder pegar varias llaves. Los campos tipo **Password** quedan cifrados y aparecen como `$encrypted$`.

B · Carga por API

```
AWX=https://awx.empresa.local
TOKEN=<token personal>     # Users → su usuario → Tokens → Add (scope Write)
ID=$(curl -sk -H "Authorization: Bearer $TOKEN" \
  "$AWX/api/v2/job_templates/?name=USUARIOS%20|%201%20-%20Crear%20usuarios" | jq '.results[0].id')

curl -sk -X POST -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  --data @awx/encuestas/01_crear_usuarios.json "$AWX/api/v2/job_templates/$ID/survey_spec/"

curl -sk -X PATCH -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  --data '{"survey_enabled": true}' "$AWX/api/v2/job_templates/$ID/"
```

Repita con los otros tres archivos JSON y sus plantillas.

C · Todo como código

```
ansible-galaxy collection install awx.awx
export CONTROLLER_HOST=https://awx.empresa.local
export CONTROLLER_OAUTH_TOKEN=<token>
ansible-playbook awx/configurar_awx.yml \
  -e awx_repo_url=https://git.empresa.local/infra/gestion_usuarios.git \
  -e awx_credencial_maquina="Linux - ansible_test" \
  -e awx_credencial_scm="Git - gestion_usuarios" \
  -e awx_credencial_smtp="SMTP - Correo corporativo"
```

Las credenciales Machine y Source Control deben existir antes. El playbook se puede repetir sin problema: actualiza lo que haya cambiado.

Paso 7 · Permisos

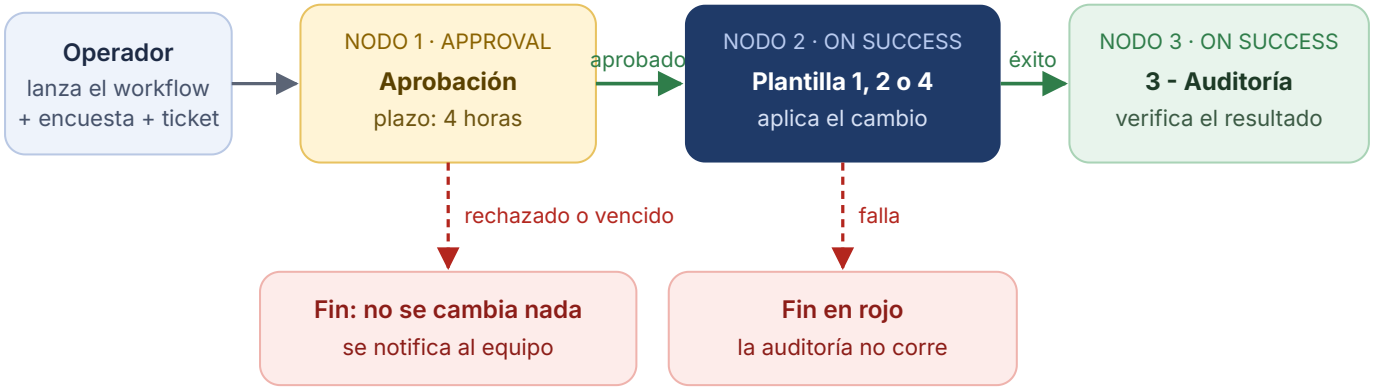
En **Access → Teams** cree los equipos y asígneles roles en la pestaña **Access** de cada plantilla:

Equipo	Plantillas	Rol
Soporte Linux - Operadores	3 directamente; 1, 2 y 4 **solo a través de los workflows** del Paso 8	Execute
Seguridad - Auditores	3	Execute
Infraestructura - Aprobadores	Workflows del Paso 8	Approve
Infraestructura - Administradores	Todas, más proyecto e inventarios	Admin

> **Si todavía no usa workflows**
> Mientras crea los workflows del Paso 8 puede dar a los operadores el rol Execute en las plantillas 1, 2 y 4. Cuando los workflows estén listos, retíreselo: así ningún cambio se ejecuta sin aprobación.

Paso 8 · Workflows con aprobación

Un **workflow** encadena varias plantillas. Aquí se usa para que los cambios sensibles (claves, altas y bajas) **esperen la aprobación de un responsable** antes de ejecutarse y, al terminar, se verifiquen con una auditoría automática. Se crean tres, todos con la misma forma:



Workflow	Nodo de aprobación	Plantilla que protege	Encuesta (JSON)
USUARIOS | WF - Cambio de contraseña con aprobación	Aprobar cambio de contraseña	2 - Cambiar contraseña	`wf_cambio_contrasena_aprobacion`
USUARIOS | WF - Crear usuarios con aprobación	Aprobar creación de usuarios	1 - Crear usuarios	`wf_crear_usuarios_aprobacion`
USUARIOS | WF - Bloquear o eliminar con aprobación	Aprobar bloqueo o eliminación	4 - Bloquear o eliminar	`wf_bloquear_eliminar_aprobacion`

> **Cómo viajan las respuestas**
> El operador llena la encuesta **del workflow**, no la de la plantilla. AWX pasa esas respuestas a todos los nodos: la plantilla de cambio recibe sus datos y la auditoría recibe el mismo «objetivo», así que verifica exactamente los servidores que se modificaron. Por eso la encuesta del workflow repite las preguntas de su plantilla, con los mismos nombres de variable, más el ticket.

8.1 Crear el workflow

El ejemplo es el de cambio de contraseña. Los otros dos se crean igual, cambiando los nombres y la plantilla de la tabla anterior.

1. Menú **Resources → Templates → Add → Add workflow template**.
2. Llene los datos generales y pulse **Save**:

Campo en AWX	Qué poner
Name	`USUARIOS | WF - Cambio de contraseña con aprobación`
Description	Aprobación → cambio de contraseña → auditoría de verificación
Organization	La misma de las plantillas
Inventory	Vacío (cada nodo usa el inventario de su plantilla)
Limit / Source control branch / Labels	Vacíos
Options	☐ Enable Concurrent Jobs · ☐ Enable Webhook

Al guardar se abre el **Workflow Visualizer**, con un único botón **Start**.

8.2 Nodo 1 · Aprobación

1. Pulse **Start**. En **Node Type** elija **Approval**.
2. Complete los campos y pulse **Save**:

Campo en AWX	Qué poner
Name	`Aprobar cambio de contraseña`
Description	Revise el ticket, el objetivo y los usuarios antes de aprobar.
Timeout	**4 horas** (240 minutos). Si nadie aprueba en ese tiempo, el nodo vence y el workflow termina **sin hacer cambios**. Use 0 solo si quiere que espere indefinidamente.

8.3 Nodo 2 · La plantilla de cambio

1. Pase el mouse sobre el nodo de aprobación y pulse el **+** que aparece a su derecha.
2. En **Run** elija **On Success**. Para una aprobación, «éxito» significa **aprobado**.
3. En **Node Type** elija **Job Template**, seleccione `USUARIOS | 2 - Cambiar contraseña` y pulse **Next**.
4. Si AWX muestra pasos de **Prompt** (inventario, límite, encuesta), déjelos como están y pulse **Next** hasta **Save**. Los datos llegan desde la encuesta del workflow.

8.4 Nodo 3 · Auditoría de verificación

1. Pulse el **+** del nodo «USUARIOS | 2 - Cambiar contraseña».
2. **Run: On Success** → **Node Type: Job Template** → `USUARIOS | 3 - Auditoría` → **Save**.
3. No agregue nodos en «On Failure» ni en el rechazo: si la aprobación se rechaza o el cambio falla, el workflow termina en rojo y la notificación avisa (8.7).

4 Pulse **Save** arriba a la derecha del visualizador. Debe verse: **Start → Aprobar cambio de contraseña → USUARIOS | 2 - Cambiar contraseña → USUARIOS | 3 - Auditoría**, con líneas verdes (On Success) entre ellos.

> **La auditoría usa sus valores por defecto**
> En el workflow, la auditoría corre con UID mínimo 1000 y salida resumida, y sin correo salvo que agregue esas preguntas a la encuesta del workflow. Su objetivo es comprobar el resultado: en el cambio de clave de root, la línea *«root: … caduca nunca»* de cada servidor confirma que se aplicó la política.

8.5 Encuesta del workflow

En el workflow, pestaña **Survey**. Cárguela de una de estas dos formas y active **Survey Enabled**:

Forma	Cómo
A mano	Primero la pregunta **Ticket o justificación** (variable `ticket_cambio`, texto, obligatoria, mínimo 3 caracteres). Después, las mismas preguntas de la plantilla que protege, en el mismo orden y con los **mismos nombres de variable** (Anexo A).
Por API	Igual que en el Paso 6, pero con el endpoint de workflows (comandos abajo).

```
ID=$(curl -sk -H "Authorization: Bearer $TOKEN" \
  "$AWX/api/v2/workflow_job_templates/?name=USUARIOS%20|%20WF%20-
%20Cambio%20de%20contrase%C3%B1a%20con%20aprobaci%C3%B3n" \
  | jq '.results[0].id')

curl -sk -X POST -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  --data @awx/encuestas/wf_cambio_contrasena_aprobacion.json \
  "$AWX/api/v2/workflow_job_templates/$ID/survey_spec/"

curl -sk -X PATCH -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  --data '{"survey_enabled": true}' "$AWX/api/v2/workflow_job_templates/$ID/"
```

> **Mantenga las dos encuestas iguales**
> Si cambia una pregunta de la plantilla 1, 2 o 4, cambie también la de su workflow. Si lo hace con `awx/generar_encuestas.py`, ambos JSON se regeneran juntos; luego recárguelos (o ejecute `configurar_awx.yml`).

8.6 Permisos del workflow

En el workflow, pestaña **Access → Add**:

Equipo	Rol en el workflow	Para qué
Soporte Linux - Operadores	**Execute**	Lanzar el workflow y llenar la encuesta.
Infraestructura - Aprobadores	**Approve**	Aprobar o rechazar el nodo de aprobación. Con este rol no pueden lanzarlo ni editarlo.
Infraestructura - Administradores	Admin	Editar nodos y encuesta.

Cierre el atajo

Después de crear los workflows, **retire a los operadores el rol Execute directo** sobre las plantillas 1, 2 y 4 (pestaña Access de cada plantilla). Para lanzar un workflow no necesitan permiso sobre las plantillas que contiene. Si conservan el Execute directo, pueden saltarse la aprobación.

Separación de funciones

Quien lanza no debería aprobar su propio cambio. AWX no lo impide por sí solo: no ponga a las mismas personas en el equipo de operadores y en el de aprobadores.

8.7 Notificaciones

1. Cree (o reutilice) una plantilla de notificación en **Administration → Notifications → Add**: tipo Email, Microsoft Teams o Slack, según lo que usen los aprobadores.
2. En el workflow, pestaña **Notifications**, active para esa notificación: **Approval** (avisa a los aprobadores cuando hay algo pendiente, aprobado, rechazado o vencido), **Failure** y **Success**.

8.8 Crearlos automáticamente

`awx/configurar_awx.yml` crea los tres workflows con sus nodos, el plazo de 4 horas (`awx_aprobacion_timeout`) y sus encuestas. Los permisos (8.6) y las notificaciones (8.7) se asignan a mano.

6 Uso diario: cómo diligenciar cada opción

Ejemplos reales de cómo llenar cada encuesta. Las preguntas que no aparecen se dejan con su valor por defecto.

Opción 1 · Crear usuarios

Ejemplo A · Tres técnicos de soporte con contraseña

Lanzar: USUARIOS | 1 - Crear usuarios

Cuentas personales, clave temporal que deben cambiar al entrar

Servidores o grupo del inventario *	`GEOPOS`
Método de acceso *	**contrasena** ▾
Origen de la lista de usuarios *	**encuesta** ▾

Usuarios (si origen = encuesta)	`jperez;Juan Pérez`
	`mgomez;María Gómez`
	`lrojas;Luis Rojas`

Grupo primario	`soporte`
Contraseña inicial (método contrasena)	••••••••••••
Forzar cambio de contraseña en el primer ingreso *	**SI** ▾
Nivel de sudo *	**ninguno** ▾

Las 9 preguntas restantes se dejan con su valor por defecto.

Lanzar ▸

Formato de la lista: una línea por usuario, `usuario;Nombre completo;grupo`. El grupo es opcional: si se omite se usa el «Grupo primario» de la encuesta, y si ese también está vacío, un grupo con el mismo nombre del usuario.

Ejemplo B · Toda una zona desde el catálogo

Origen = `catalogo`, Catálogo = `usuarios_zona_centro`, Método = `contrasena`, contraseña inicial y forzar cambio = `SI`. El catálogo ya trae `grupo: soporte`.

Ejemplo C · Administrador con llave SSH y sudo completo

Lanzar: USUARIOS | 1 - Crear usuarios

Acceso solo por llave: jcantor ya tiene llave en keys/, se reutiliza

Servidores o grupo del inventario *	PPU
Método de acceso *	llave
Origen de la lista de usuarios *	encuesta
Usuarios (si origen = encuesta)	jcantor;Administrador Linux - Amaris
Origen de la llave (método llave)	keys
⚠ RENOVAR LLAVE (opción crítica) *	NO
Nivel de sudo *	completo

Las 10 preguntas restantes se dejan con su valor por defecto.

Lanzar ▸

Con método llave

La contraseña queda bloqueada y sin caducidad, y el sudo se configura **sin contraseña** automáticamente (la cuenta no tiene clave que digitar). Como `jcantor` ya tiene llave en `keys/`, se instala esa misma: el usuario entra con la llave que ya conoce.

Ejemplo D · Usuario de monitoreo con sudo limitado

Lanzar: USUARIOS | 1 - Crear usuarios

Solo puede ejecutar dos comandos como root

Servidores o grupo del inventario *	PPU_GEOPOS
Método de acceso *	contrasena ▾
Usuarios (si origen = encuesta)	jmonitor;Usuario de monitoreo
Contraseña inicial (método contrasena)	••••••••••••
Nivel de sudo *	limitado ▾
sudo sin contraseña (NOPASSWD) *	SI ▾
Comandos sudo permitidos (nivel limitado)	/usr/bin/systemctl restart httpd /usr/bin/tail -n 200 /var/log/messages

Las 10 preguntas restantes se dejan con su valor por defecto.

Lanzar ▸

> **Comandos que nunca debe dar por sudo**
> Escriba cada comando con **ruta absoluta**. Evite `less`, `more`, `vi`, `journalctl`, `systemctl status`, `find`, `awk`, `python`, `tar` o `cp`: todos permiten abrir una shell de root. La auditoría los marca como hallazgo ALTO.

Llaves SSH: reutilizar, generar o renovar

Con «Origen de la llave = **keys**» (valor por defecto) cada usuario tiene **una sola llave**, guardada en `keys/`, y es la misma en todos los servidores:

Situación del usuario	«Renovar llave»	Qué pasa
Ya tiene llave en `keys/`	NO	**Se reutiliza** esa llave. No se genera nada.
No tiene llave en `keys/`	NO / SI	Se **genera** una (RSA 4096, con `.ppk` si hay puttygen), se guarda en `keys/` y se **publica en Git** antes de tocar servidores.
Ya tiene llave en `keys/`	**SI**	Se genera una **nueva**. La anterior se mueve a `keys/_anteriores/<usuario>_<fecha>`. En los servidores del objetivo se **retira la llave anterior** y se instala la nueva, también si la cuenta ya existía.

«Renovar llave» es una opción crítica

Úsela solo si la llave se perdió o pudo quedar expuesta. Afecta únicamente a los servidores del «objetivo» de esa ejecución: en los demás servidores el usuario seguirá entrando **solo con la llave anterior**, que ya no estará en `keys/` sino en `keys/_anteriores/`. Para renovar por completo, ejecute con un objetivo que incluya **todos** los servidores donde existe el usuario. Después entregue la nueva llave privada al usuario por un canal seguro.

Origen «encuesta»

Sirve cuando la persona genera su propio par y entrega solo la pública: se pega en la encuesta (`usuario;ssh-rsa AAAA...`) y no se guarda en `keys/`. «Renovar llave» no aplica en este caso.

¿Y si el usuario ya existe?

Con «Actualizar usuarios que ya existen = NO» (valor por defecto) se omite y se informa. Con **SI** se le agregan grupos, llave y regla sudo; nunca se cambian su contraseña, home, shell, UID ni grupo primario.

Opción 2 · Cambiar contraseña

Ejemplo · Clave de root sin caducidad

Lanzar: USUARIOS | 2 - Cambiar contraseña

Caso típico: el hardening vence la clave de root cada 90 días

Servidores o grupo del inventario *	PPU_GEOPOS
Usuario(s) *	root
Nueva contraseña *	••••••••••••
Confirmar contraseña *	••••••••••••
Contraseña sin caducidad (nunca expira) *	SI
Quitar también la fecha de expiración de la cuenta *	NO
Forzar cambio en el próximo inicio de sesión *	NO
Desbloquear la cuenta *	SI
Permitir cuentas de servicio (UID 1-999) *	NO

Lanzar ▸

Qué hace en cada servidor

1. **Cambia la clave** con `chpasswd`, que la recibe por entrada estándar: no aparece en procesos ni en logs, y el servidor genera el hash con su propio algoritmo (SHA-512 o yescrypt).
2. **Quita la caducidad** con `chage -m 0 -M 99999 -I -1`. Desde ese momento `chage -l root` muestra *Password expires: never*, aunque el hardening tenga `PASS_MAX_DAYS 90`. Si la cuenta ya estaba así, no hace nada.
3. **Desbloquea**: reinicia los intentos fallidos (`faillock` o `pam_tally2`) y quita una expiración de cuenta ya vencida.
4. **Verifica** en `/etc/shadow` que la fecha de último cambio sea la de hoy y muestra el resultado de `chage -l`.

Pregunta	Cuándo responder SI
Contraseña sin caducidad	Cuentas críticas (root, cuentas de aplicación) que no pueden vencer. Es la opción por defecto.
Quitar también la fecha de expiración de la cuenta	Solo si la cuenta tenía una fecha de vencimiento y debe quedar permanente.
Forzar cambio en el próximo inicio	Cuentas personales donde el usuario debe poner su propia clave. No se puede junto con «sin caducidad» ni sobre root.
Permitir cuentas de servicio	Casi nunca. Habilita cambiar claves de cuentas con UID entre 1 y 999.

Después de cambiar la clave de root

Regístrela en la bóveda de contraseñas corporativa. Si la clave no cumple la política de complejidad del servidor, el job falla en ese servidor con el mensaje del sistema, sin mostrar la clave. Si cambió la clave de la cuenta con la que se conecta AWX y la credencial usa contraseña, actualícela (el job avisa).

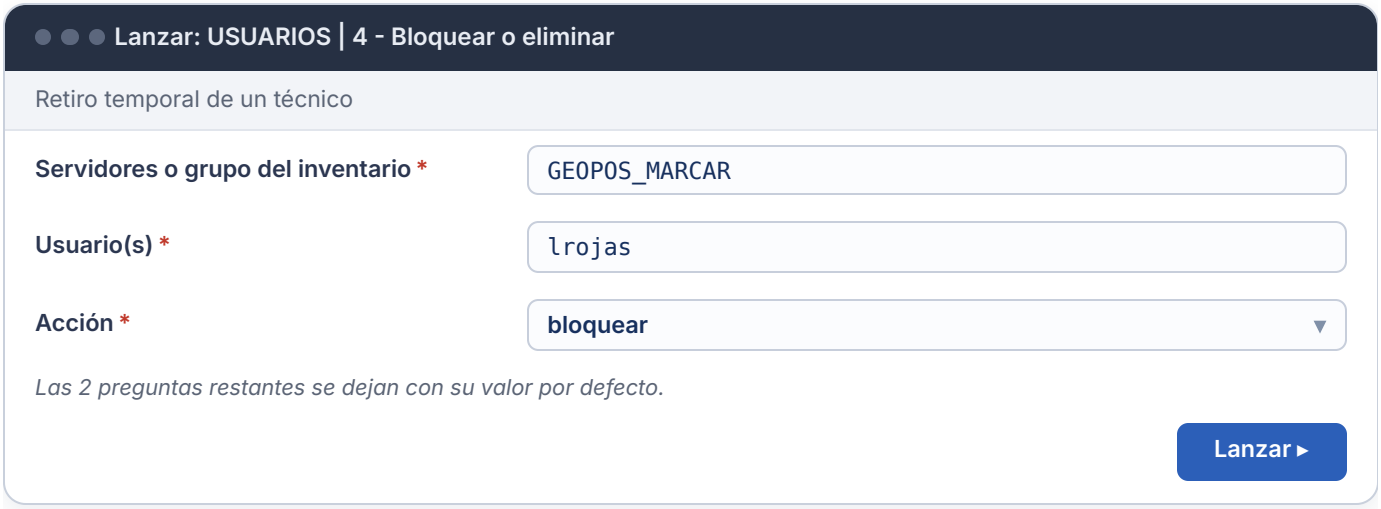
Opción 3 · Auditoría



Conserve los reportes

AWX borra el contenedor al terminar el job. Si no activa el correo ni «Copiar reportes a servidor», los archivos HTML y CSV se pierden (la salida del job sí queda).

Opción 4 · Bloquear o eliminar



Acción	Qué hace	Cómo revertir
bloquear	Bloquea la clave, expira la cuenta (impide también entrar con llave) y retira su regla sudo. Conserva el home y sus archivos.	Opción 2 con «Desbloquear = SI» y una clave nueva. La regla sudo se vuelve a crear con la opción 1.
eliminar	Respalda el home en `/var/backups/usuarios_eliminados/`, retira sudo y borra la cuenta y su home. Exige escribir `CONFIRMAR`.	Crear de nuevo y restaurar el respaldo.

Lanzar y aprobar un workflow

En producción los cambios se lanzan desde los workflows del Paso 8. Para el operador solo cambian dos cosas: el formulario empieza con el **ticket** y, después de lanzar, hay que esperar la aprobación.

Operador · Lanzar

1. **Resources → Templates**, busque `USUARIOS | WF - Cambio de contraseña con aprobación` y pulse el cohete (**Launch**).
2. Llene la encuesta, que tiene las mismas preguntas de la opción 2 más el ticket, y pulse **Launch**.
3. El workflow queda en **Pending approval**. No cierre el ticket hasta ver el workflow en verde.

Lanzar: USUARIOS | WF - Cambio de contraseña con aprobación

Encuesta del workflow: el ticket va primero

Ticket o justificación *	RFC-2026-0458 · Rotación trimestral de la clave de root en POS
Servidores o grupo del inventario *	PPU_GEOPOS
Usuario(s) *	root
Nueva contraseña *	••••••••••••
Confirmar contraseña *	••••••••••••
Contraseña sin caducidad (nunca expira) *	SI ▾
Desbloquear la cuenta *	SI ▾

Las 3 preguntas restantes se dejan con su valor por defecto.

Lanzar ▸

Aprobador · Revisar y aprobar

1. Llega la notificación, o aparece un número en la campana superior. Abra **Views → Workflow Approvals**.
2. Pulse el nombre del workflow para ver sus **Details**. En **Variables** revise el ticket, el `objetivo` y los usuarios. La contraseña aparece como `$encrypted$`: nadie la ve.
3. Vuelva a la lista, marque la fila y pulse **Approve** o **Deny**. Con Deny el workflow termina sin cambiar nada.



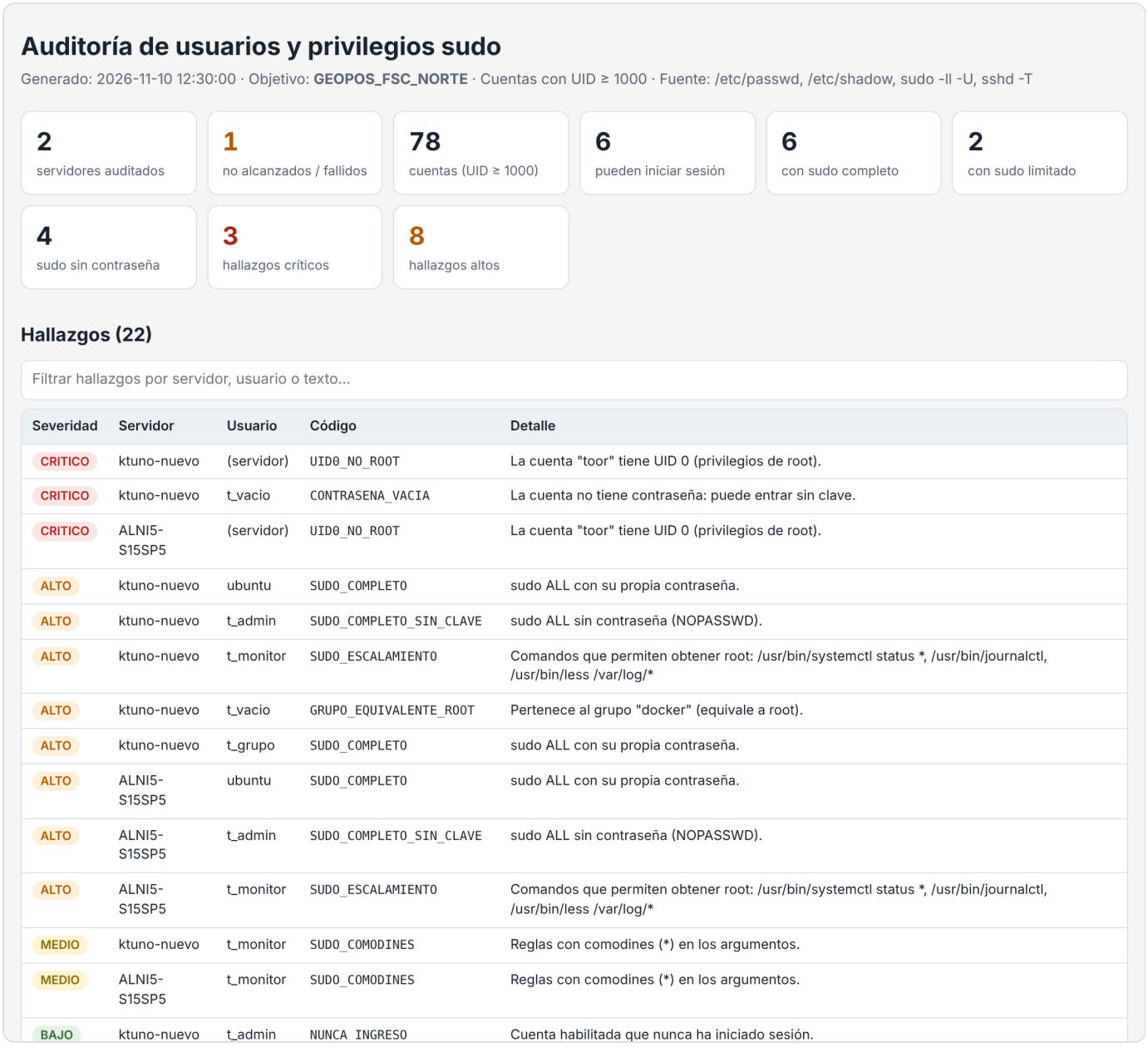
Después de aprobar

Corre la plantilla de cambio y, si termina bien, la auditoría de los mismos servidores. El workflow completo queda en **Views → Jobs** con quién lo lanzó, quién lo aprobó y la salida de cada nodo: es el registro del cambio para el ticket.

7 Cómo leer la auditoría

Dónde ver los resultados

Lugar	Qué contiene
Salida del job	Una tabla por servidor y un consolidado con todas las cuentas que tienen sudo y los hallazgos críticos y altos.
Artifacts del job	Conteos por servidor (`auditoria_usuarios.servidores`), útiles para encadenar en un Workflow.
Correo	Reporte HTML y CSV adjuntos, si se activó.
Servidor de reportes	HTML y CSV en `/var/reportes/auditoria_usuarios/` , si se indicó «Copiar reportes a servidor».



Auditoría de usuarios y privilegios sudo

Generado: 2026-11-10 12:30:00 · Objetivo: **GEOPOS_FSC_NORTE** · Cuentas con UID ≥ 1000 · Fuente: /etc/passwd, /etc/shadow, sudo -ll -U, sshd -T

2	1	78	6	6	2
servidores auditados	no alcanzados / fallidos	cuentas (UID ≥ 1000)	pueden iniciar sesión	con sudo completo	con sudo limitado

4	3	8
sudo sin contraseña	hallazgos críticos	hallazgos altos

Hallazgos (22)

Filtrar hallazgos por servidor, usuario o texto...

Severidad	Servidor	Usuario	Código	Detalle
CRITICO	ktuno-nuevo	(servidor)	UID0_NO_ROOT	La cuenta "toor" tiene UID 0 (privilegios de root).
CRITICO	ktuno-nuevo	t_vacio	CONTRASENA_VACIA	La cuenta no tiene contraseña: puede entrar sin clave.
CRITICO	ALNI5-S15SP5	(servidor)	UID0_NO_ROOT	La cuenta "toor" tiene UID 0 (privilegios de root).
ALTO	ktuno-nuevo	ubuntu	SUDO_COMPLETO	sudo ALL con su propia contraseña.
ALTO	ktuno-nuevo	t_admin	SUDO_COMPLETO_SIN_CLAVE	sudo ALL sin contraseña (NOPASSWD).
ALTO	ktuno-nuevo	t_monitor	SUDO_ESCALAMIENTO	Comandos que permiten obtener root: /usr/bin/systemctl status *, /usr/bin/journalctl, /usr/bin/less /var/log/*
ALTO	ktuno-nuevo	t_vacio	GRUPO_EQUIVALENTE_ROOT	Pertenece al grupo "docker" (equivale a root).
ALTO	ktuno-nuevo	t_grupo	SUDO_COMPLETO	sudo ALL con su propia contraseña.
ALTO	ALNI5-S15SP5	ubuntu	SUDO_COMPLETO	sudo ALL con su propia contraseña.
ALTO	ALNI5-S15SP5	t_admin	SUDO_COMPLETO_SIN_CLAVE	sudo ALL sin contraseña (NOPASSWD).
ALTO	ALNI5-S15SP5	t_monitor	SUDO_ESCALAMIENTO	Comandos que permiten obtener root: /usr/bin/systemctl status *, /usr/bin/journalctl, /usr/bin/less /var/log/*
MEDIO	ktuno-nuevo	t_monitor	SUDO_COMODINES	Reglas con comodines (*) en los argumentos.
MEDIO	ALNI5-S15SP5	t_monitor	SUDO_COMODINES	Reglas con comodines (*) en los argumentos.
BAJO	ktuno-nuevo	t_admin	NUNCA_INGRESO	Cuenta habilitada que nunca ha iniciado sesión.

Parte superior del reporte HTML (ejemplo con cuentas de prueba). Más abajo siguen la tabla de cuentas con sudo, con sus comandos y el archivo que da el permiso, y el detalle por servidor. Cada tabla tiene un buscador. El CSV abre directo en Excel (separado por punto y coma).

Niveles de sudo

El permiso se obtiene con `sudo -ll -U <usuario>`, que devuelve lo que la cuenta **realmente puede hacer**, venga de una línea propia, de un grupo (`%wheel`, `%admins`), de un alias, de `/etc/sudoers.d` o de LDAP/SSSD.

Nivel	Significado
COMPLETO	Alguna regla concede `ALL`: puede hacer cualquier cosa como root. Las excepciones del tipo `ALL, !/bin/su` no lo limitan en la práctica.

Nivel	Significado
LIMITADO	Solo los comandos listados. Si alguno permite abrir una shell (editores, paginadores, intérpretes), se marca en rojo y genera hallazgo ALTO.
NINGUNO	No puede usar sudo.

> **Caso SUSE: todos aparecen con sudo COMPLETO**
> SLES trae por defecto `Defaults targetpw` y `ALL ALL=(ALL) ALL` : cualquier usuario puede usar sudo, pero debe digitar la **clave de root**. El reporte lo muestra como «Clave exigida: root (targetpw)» y lo clasifica MEDIO, no ALTO.

Hallazgos

Severidad	Código	Significado
CRÍTICO	`UID0_NO_ROOT`	Otra cuenta además de root tiene UID 0 (es root con otro nombre).
CRÍTICO	`CONTRASENA_VACIA`	La cuenta entra sin contraseña.
ALTO	`SUDO_COMPLETO_SIN_CLAVE`	Sudo ALL sin contraseña (NOPASSWD).
ALTO	`SUDO_COMPLETO`	Sudo ALL con su propia contraseña.
ALTO	`SUDO_ESCALAMIENTO`	Sudo limitado, pero a comandos que permiten obtener root.
ALTO	`GRUPO_EQUIVALENTE_ROOT`	Pertenece a docker, lxd, disk u otro grupo que equivale a root.
ALTO	`HASH_DEBIL`	Clave guardada con MD5 o DES.
ALTO	`ROOT_CLAVE_EXPIRADA`	La clave de root está vencida (use la opción 2).
MEDIO	`SUDO_COMPLETO_CLAVE_ROOT`	Sudo ALL que exige la clave de root (targetpw/rootpw).
MEDIO	`SUDO_COMODINES`	Reglas con * en los argumentos.
MEDIO	`CONTRASENA_EXPIRADA`	La clave del usuario está vencida.
MEDIO	`CUENTA_INACTIVA`	Cuenta habilitada sin ingresar hace más de N días.
MEDIO	`SSH_ROOT_PERMITIDO`	sshd permite entrar directo como root (PermitRootLogin yes).
BAJO	`NUNCA_INGRESO`	Cuenta habilitada que nunca ha iniciado sesión.
BAJO	`CONTRASENA_SIN_CADUCIDAD`	Cuenta personal con clave que no caduca.
INFO	`Otros`	SUDO_NEGACIONES, CUENTA_EXPIRADA, LLAVE_CON_OPCIONES, ROOT_CLAVE_CADUCA, SUDOERS_SIN_INCLUDEDIR.

Otras columnas útiles

Columna	Qué indica
Acceso	Si la cuenta puede iniciar sesión hoy: shell interactiva, permitida por AllowUsers/AllowGroups de SSH, no expirada y con clave activa o llave SSH.

Columna	Qué indica
Contraseña	**ACTIVA** (utilizable) · **BLOQUEADA** · **SIN_CLAVE** (nunca se asignó; solo entra con llave) · **VACIA** (entra sin clave).
Caduca	Fecha en que vence la clave, o «nunca».
Llaves	Cantidad de llaves SSH autorizadas (al pasar el mouse se ve el comentario de cada una).
Último acceso	Según `lastlog`. «NUNCA» si nunca ha entrado.

8 Seguridad y buenas prácticas

- **Contraseñas solo por encuesta.** Nunca las escriba en archivos del repositorio. Desde la línea de comandos use un archivo cifrado con `ansible-vault`.
- **No suba la verbosidad** de las plantillas 1 y 2 por encima de 2 en producción: aunque las tareas sensibles están ocultas (`no_log`), es mejor no arriesgar.
- **El repositorio guarda llaves privadas.** Manténgalo privado, con acceso solo para infraestructura, y use un token de escritura exclusivo para AWX. Entregue cada llave privada a su dueño por un canal seguro (nunca por correo sin cifrar).
- **«Renovar llave» solo cuando haga falta** (llave perdida o expuesta), y con un objetivo que incluya todos los servidores donde existe el usuario.
- **«Nunca caduca» es una excepción a la política.** Documente para qué cuentas se aplica (root y cuentas de aplicación) y ejecute la auditoría periódicamente para revisarlas.
- **Cambios con aprobación.** En producción, los operadores lanzan las opciones 1, 2 y 4 solo mediante los workflows del Paso 8, y nunca aprueban sus propios cambios.
- **Pruebe antes de un cambio masivo.** Lance la plantilla con **Job Type = Check** sobre un servidor: muestra qué cambiaría sin aplicarlo. Los cambios de contraseña no se simulan, y en la opción 1 las tareas que dependen de una cuenta aún no creada pueden marcar error en ese modo (en la ejecución real no ocurre).
- **Auditoría periódica.** Programe la plantilla 3 (**Schedules**) cada mes con envío por correo.

9 Solución de problemas

Síntoma	Causa y solución
«Could not match supplied host pattern» y el job no hace nada	El valor de «Servidores o grupo» no existe en el inventario elegido. Revise mayúsculas (`OCI_PEOPLE_PRD` ; la lista está en el Anexo B) y que el inventario esté sincronizado.
Falla en una tarea «Preparar | Validar...»	La encuesta tiene un dato inválido; el mensaje dice cuál. No se tocó ningún servidor.
«Missing sudo password»	La cuenta de conexión pide contraseña para sudo: agréguela en la credencial (Privilege Escalation Password).
«chpasswd falló ... BAD PASSWORD»	La clave no cumple la política de complejidad o de historial del servidor. Use una más fuerte.
«couldn't resolve module/action ansible.posix.authorized_key»	No se instalaron las colecciones: sincronice el proyecto y revise el acceso a Galaxy o Automation Hub.
Aviso «sudoers no lee /etc/sudoers.d»	Falta `#includedir /etc/sudoers.d` en `/etc/sudoers` . La regla se creó pero no aplica hasta agregar esa línea.
El usuario con llave no puede entrar	Revise que su grupo esté permitido en AllowGroups/AllowUsers de sshd (la auditoría lo muestra en «Acceso»). El proyecto agrega el grupo `ssh_access` automáticamente si existe.
«Hay que generar llaves ... el contenedor se borra»	La plantilla 1 no tiene la credencial «Git - publicar llaves» (Paso 2.4). Es intencional: sin ella la llave se perdería y la próxima vez se generaría otra. No se modificó ningún servidor.
«No se pudieron publicar las llaves ... push rechazado»	El token no tiene permiso de escritura o la rama está protegida. Corrija y vuelva a lanzar: no se modificó ningún servidor.
El workflow queda en «Pending approval»	Espera a un aprobador. Los aprobadores lo ven en **Views → Workflow Approvals** (y en la campana superior). Si nadie lo ve, revise el rol Approve (8.6) y la notificación Approval (8.7).
El nodo de aprobación terminó en «Timed out»	Pasó el plazo de 4 horas sin respuesta. No se cambió nada: lance de nuevo el workflow.
Un operador ejecutó un cambio sin aprobación	Todavía tiene Execute directo sobre la plantilla 1, 2 o 4. Retírelo (8.6).
Solaris 11	Programado pero no probado en equipo real; valide en uno de prueba. Las cuentas de solo llave se dejan con `passwd -N` , porque en Solaris `passwd -l` también bloquea el acceso por llave.

10 Uso por línea de comandos

Todo lo anterior también funciona sin AWX, pasando las respuestas de la encuesta como variables `-e`.

```
ansible-galaxy collection install -r collections/requirements.yml

# Auditoría
ansible-playbook -i inventario/hosts.ini 03_auditoria_usuarios.yml -e objetivo=GEOPOS

# Crear desde catálogo (contraseña desde archivo cifrado)
ansible-playbook -i inventario/hosts.ini 01_crear_usuarios.yml \
  -e objetivo=GEOPOS -e crear_origen=catalogo -e crear_catalogo=usuarios_zona_centro \
  -e @secretos.yml --ask-vault-pass

# Clave de root sin caducidad
ansible-playbook -i inventario/hosts.ini 02_cambiar_contrasena.yml \
  -e objetivo=PPU_GEOPOS -e contrasena_usuarios=root -e contrasena_nunca_caduca=SI \
  -e @secretos.yml --ask-vault-pass
```

`secretos.yml` se crea con `ansible-vault create secretos.yml` y define `crear_contrasena_inicial`, `contrasena_nueva` y `contrasena_confirmacion`. Los reportes de la auditoría quedan en `reportes/`.

A Todas las preguntas de las encuestas

Generado desde `awx/generar_encuestas.py`, la misma fuente de los archivos JSON. La columna «Variable» debe escribirse exactamente igual en AWX.

USUARIOS | 1 - Crear usuarios

#	Pregunta y ayuda (* = obligatoria)	Variable	Tipo	Por defecto
1	**Servidores o grupo del inventario** * Grupo (ej. GEOPOS, PPU, OCI_PEOPLE_PRD), servidor, o patrón Ansible: grupo1:grupo2, grupo:!servidor, srv1,srv2.	`objetivo`	Texto	—
2	**Método de acceso** * contrasena: cuenta con contraseña inicial. llave: solo acceso con llave SSH (la contraseña queda bloqueada).	`crear_metodo_acceso`	Opción múltiple contrasena / llave	`contrasena`
3	**Origen de la lista de usuarios** * encuesta: los usuarios se escriben abajo. catalogo: se toman de un archivo de vars/catalogos/ del proyecto.	`crear_origen`	Opción múltiple encuesta / catalogo	`encuesta`
4	**Usuarios (si origen = encuesta)** Una línea por usuario: usuario;Nombre completo;grupo(opcional) Ej: jperez;Juan Pérez;soporte	`crear_usuarios_texto`	Área de texto	—
5	**Catálogo (si origen = catalogo)** Archivo de vars/catalogos/. Para agregar opciones, cree el archivo y añádalo a esta lista.	`crear_catalogo`	Opción múltiple usuarios_zona_norte / usuarios_zona_centro / usuarios_individuales / usuarios_llave_admin_linux / usuarios_llave_servicio	—
6	**Grupo primario** Vacío = un grupo con el mismo nombre de cada usuario. Se crea si no existe. El catálogo puede definirlo por usuario.	`crear_grupo_primario`	Texto	—
7	**Grupos secundarios** Separados por coma. Solo se agregan los que ya existen en el servidor (los demás se informan).	`crear_grupos_secundarios`	Texto	—
8	**Contraseña inicial (método contrasena)** Obligatoria con método contrasena. Solo se asigna a cuentas nuevas. AWX la guarda cifrada.	`crear_contrasena_inicial`	Contraseña	—
9	**Forzar cambio de contraseña en el primer ingreso** * Recomendado SI para cuentas personales.	`crear_forzar_cambio`	Opción múltiple SI / NO	`SI`

#	Pregunta y ayuda (* = obligatoria)	Variable	Tipo	Por defecto
10	**Origen de la llave (método llave)** keys: usa la llave del usuario guardada en keys/ del proyecto; si no tiene, genera una nueva y la guarda ahí. encuesta: usa la llave pública pegada abajo.	`crear_llave_origen`	Opción múltiple keys / encuesta	`keys`
11	**Llaves públicas (origen de llave = encuesta)** Una línea por usuario: usuario;ssh-rsa AAAA... comentario	`crear_llaves_texto`	Área de texto	—
12	**⚠ RENOVAR LLAVE (opción crítica)** * SI genera una llave NUEVA aunque el usuario ya tenga una en keys/. La anterior se archiva y se retira de los servidores del objetivo. En servidores NO incluidos en el objetivo el usuario seguirá entrando solo con la llave anterior. Deje NO salvo pérdida o compromiso de la llave.	`crear_renovar_llave`	Opción múltiple SI / NO	`NO`
13	**Servidor custodio de llaves (opcional)** Servidor del inventario donde además se copian las llaves nuevas (/root/llaves_generadas). En AWX es obligatorio solo si la plantilla no tiene la credencial "Git - publicar llaves".	`crear_llaves_custodio_host`	Texto	—
14	**Nivel de sudo** * ninguno | completo (ALL) | limitado (solo los comandos indicados abajo).	`crear_sudo_nivel`	Opción múltiple ninguno / completo / limitado	`ninguno`
15	**sudo sin contraseña (NOPASSWD)** * Con método llave se aplica SI automáticamente (la cuenta no tiene contraseña).	`crear_sudo_nopasswd`	Opción múltiple SI / NO	`NO`
16	**Comandos sudo permitidos (nivel limitado)** Un comando por línea, con ruta absoluta. Ej: /usr/bin/systemctl restart httpd /usr/bin/tail -f /var/log/messages	`crear_sudo_comandos`	Área de texto	—
17	**Actualizar usuarios que ya existen** * NO: se omiten. SI: se les agregan grupos, llave y regla sudo (no se cambia su contraseña, home ni shell).	`crear_actualizar_existentes`	Opción múltiple SI / NO	`NO`

USUARIOS | 2 - Cambiar contraseña

#	Pregunta y ayuda (* = obligatoria)	Variable	Tipo	Por defecto
1	**Servidores o grupo del inventario** * Grupo (ej. GEOPOS, PPU, OCI_PEOPLE_PRD), servidor, o patrón Ansible: grupo1:grupo2, grupo:!servidor, srv1,srv2.	`objetivo`	Texto	—

#	Pregunta y ayuda (* = obligatoria)	Variable	Tipo	Por defecto
2	**Usuario(s)** * Uno o varios separados por coma. Ej: root o root,soporte1	`contrasena_usuarios`	Texto	`root`
3	**Nueva contraseña** * AWX la guarda cifrada y no aparece en la salida del job.	`contrasena_nueva`	Contraseña	—
4	**Confirmar contraseña** * Debe coincidir con la anterior.	`contrasena_confirmacion`	Contraseña	—
5	**Contraseña sin caducidad (nunca expira)** * SI: chage -m 0 -M 99999 -I -1. Evita que el hardening haga caducar la clave (crítico para root). NO: conserva la política actual.	`contrasena_nunca_caduca`	Opción múltiple SI / NO	`SI`
6	**Quitar también la fecha de expiración de la cuenta** * SI: chage -E -1 (la cuenta queda permanente).	`contrasena_quitar_expiracion_cuenta`	Opción múltiple SI / NO	`NO`
7	**Forzar cambio en el próximo inicio de sesión** * Excluyente con "sin caducidad". No permitido para root ni cuentas protegidas.	`contrasena_forzar_cambio`	Opción múltiple SI / NO	`NO`
8	**Desbloquear la cuenta** * Reinicia intentos fallidos (faillock/pam_tally2) y quita una expiración de cuenta ya vencida.	`contrasena_desbloquear`	Opción múltiple SI / NO	`SI`
9	**Permitir cuentas de servicio (UID 1-999)** * Por seguridad NO. root (UID 0) siempre está permitido.	`contrasena_permitir_cuentas_servicio`	Opción múltiple SI / NO	`NO`

USUARIOS | 3 - Auditoría

#	Pregunta y ayuda (* = obligatoria)	Variable	Tipo	Por defecto
1	**Servidores o grupo del inventario** * Grupo (ej. GEOPOS, PPU, OCI_PEOPLE_PRD), servidor, o patrón Ansible: grupo1:grupo2, grupo:!servidor, srv1,srv2.	`objetivo`	Texto	—
2	**UID mínimo a auditar** * Por debajo de este UID son cuentas de servicio.	`auditoria_uid_minimo`	Entero	`1000`
3	**Días sin ingreso para marcar cuenta inactiva** *	`auditoria_dias_inactividad`	Entero	`90`
4	**Detalle en la salida del job** * resumido: tabla por servidor. completo: además los hallazgos de cada usuario.	`auditoria_detalle_salida`	Opción múltiple resumido / completo	`resumido`
5	**Enviar reporte por correo** * Requiere la credencial SMTP asociada a la plantilla.	`auditoria_enviar_correo`	Opción múltiple SI / NO	`NO`

#	Pregunta y ayuda (* = obligatoria)	Variable	Tipo	Por defecto
6	**Destinatarios** Separados por coma.	`auditoria_correo_destinatarios`	Texto	—
7	**Copiar reportes a servidor** Opcional. Servidor del inventario donde se dejan HTML y CSV (/var/reportes/auditoria_usuarios).	`auditoria_copiar_a_host`	Texto	—

USUARIOS | 4 - Bloquear o eliminar

#	Pregunta y ayuda (* = obligatoria)	Variable	Tipo	Por defecto
1	**Servidores o grupo del inventario** * Grupo (ej. GEOPOS, PPU, OCI_PEOPLE_PRD), servidor, o patrón Ansible: grupo1:grupo2, grupo:!servidor, srv1,srv2.	`objetivo`	Texto	—
2	**Usuario(s)** * Uno o varios separados por coma.	`eliminar_usuarios`	Texto	—
3	**Acción** * bloquear: bloquea clave y expira la cuenta (conserva el home). eliminar: borra cuenta y home.	`eliminar_accion`	Opción múltiple bloquear / eliminar	`bloquear`
4	**Respaldar home antes de eliminar** * Se guarda en /var/backups/usuarios_eliminados/ del servidor.	`eliminar_respaldar_home`	Opción múltiple SI / NO	`SI`
5	**Confirmación (solo para eliminar)** Escriba CONFIRMAR para eliminar.	`eliminar_confirmacion`	Texto	—

Encuestas de los workflows (Paso 8)

Cada workflow usa la encuesta de su plantilla con esta pregunta adicional al inicio:

#	Pregunta y ayuda (* = obligatoria)	Variable	Tipo	Por defecto
0	**Ticket o justificación** * Número de ticket / RFC y motivo del cambio. Es lo primero que revisa quien aprueba.	`ticket_cambio`	Texto	—

Workflow	Archivo de encuesta	Preguntas
USUARIOS | WF - Cambio de contraseña con aprobación	`awx/encuestas/wf_cambio_contrasena_aprobacion.json`	Ticket + las 9 de «USUARIOS | 2 - Cambiar contraseña»
USUARIOS | WF - Crear usuarios con aprobación	`awx/encuestas/wf_crear_usuarios_aprobacion.json`	Ticket + las 17 de «USUARIOS | 1 - Crear usuarios»

Workflow	Archivo de encuesta	Preguntas
USUARIOS | WF - Bloquear o eliminar con aprobación	`awx/encuestas/wf_bloquear_eliminar_aprobacion.json`	Ticket + las 5 de «USUARIOS | 4 - Bloquear o eliminar»

B Grupos del inventario

Calculado desde `inventario/hosts.ini`. El nombre del grupo es lo que se escribe en «Servidores o grupo del inventario». El inventario no tiene IP duplicadas ni nombres repetidos.

Grupo (valor para «Servidores o grupo»)	Servidores	Usuario de conexión	Observación
AZURE	1	(credencial de AWX)	
manhatan	3	`tux`	
hardening_linux	4	`tux`	
OCI_PEOPLE_PRUEBAS	9	`adminso`	
OCI_PEOPLE_PRD	10	`adminso`	
PPU	104	`ansible_test`	⚠ PASBOGDIS01: conexión local
INFRA2_0	10	`ansible_test`	
PPU_GEOPOS	59	`ansible_test`	
GEOPOS	187	`ansible_test`	
GEOPOS_MARCAR	20	(credencial de AWX)	

C Cambios frente a los proyectos anteriores

Antes	Ahora
Contraseñas `Soporte2025$` / `Soporte2026$` en texto plano	Campo cifrado de la encuesta
Llaves en `con_LLave/keys/`, solo para ese proyecto	Las mismas llaves en `keys/`, usadas por la opción 1 en cualquier servidor
Servidores fijos en cada playbook (`ktuno-nuevo`, `AZURE`, `PPU_GEOPOS`…)	Campo «Servidores o grupo» en la encuesta
Dos proyectos separados (con contraseña / con llave)	Una sola opción 1 con «Método de acceso»
Dos inventarios distintos, con 40 servidores con IP en conflicto	Un solo `inventario/hosts.ini` (INVENTARIO 2026), sin conflictos
`chage -d 0` a todos los usuarios de la zona en cada ejecución	Solo a cuentas recién creadas
`exclusive: true` en las llaves (borraba otras llaves del usuario)	Configurable; por defecto no borra
Se generaba una llave nueva en cada ejecución (o se perdía en AWX)	Se reutiliza la llave existente; solo se genera si falta o si se marca «Renovar llave», y se publica en Git
Eliminar sin respaldo ni protección	Bloqueo reversible o eliminación con respaldo, confirmación y cuentas protegidas
`desbloquear_user_pos.yml` solo para `pos`	Opción 2 con cualquier usuario
Sin auditoría	Opción 3 con reporte HTML/CSV, niveles de sudo y hallazgos